

1. Project Overview (Microservices-based E-Commerce)

We will build a **basic e-commerce system** with four core microservices:

1. Auth Service

- Handles **user registration, login, JWT token generation, and validation**.
- Stores user data in **MySQL** (users table).
- Provides token-based authentication to other services.

2. Product Service

- Manages products: CRUD operations (add, update, delete, list).
- Stores product info in **MySQL** (products table).
- Provides REST APIs for fetching product details.

3. Cart Service

- Manages user carts: add/remove items, update quantity.
- Uses **MySQL** (cart table with mapping to users & products).
- Talks to Product Service to fetch product details (price, availability).

4. Order Service

- Handles placing orders, and order history.
- Stores order details in **MySQL** (orders, order_items).
- Calls Cart Service to finalize cart items.
- Calls Product Service to update stock.

5. Payment Service

- Handles payments, uses Stripe payment for payment-service
 - Payment-service produces event to kafka topic after payment success
 - Stores order details in **MySQL** (payments).
-

2. Management & Communication

a. Service Independence

- Each service runs on a **different port**.
- Each has its own **database schema** (important for microservices – no direct DB sharing).

b. Communication

- **Synchronous**: Services communicate via **REST APIs** (e.g., Cart → Product to validate item).
- **Asynchronous**: Services communicate via **Kafka**
- **Authentication**: JWT from Auth Service is passed in HTTP headers.
- (Optional later) **Asynchronous**: Use **Kafka/RabbitMQ** for event-driven flow like "Order Placed → Notify Cart → Update Inventory."

c. API Gateway

- A **Spring Cloud Gateway** will sit in front:
 - Route requests to services (`/auth/**`, `/products/**`, `/cart/**`, `/orders/**`).
 - Centralize authentication/authorization.

d. Service Discovery

- Use **Spring Cloud Netflix Eureka** or **Consul** for service discovery.
- Each microservice registers itself → Gateway discovers them dynamically.

e. Frontend (React)

- React app interacts **only with API Gateway**, not with individual services.
 - React handles UI: login/register, product catalog, add to cart, checkout.
-

3. Prerequisite Learning Checklist

Here's the **must-know learning path** for you (trimmed to only what you need):

Java & Spring Boot

- Spring Boot basics (controllers, services, repositories).
- REST APIs (GET, POST, PUT, DELETE).
- Spring Data JPA (basic CRUD with MySQL).
- Exception handling in REST.
- Spring Security basics (JWT authentication).
- Microservices basics (what, why, how).

Microservices & Cloud Tools

- Spring Cloud Gateway (API Gateway).
- Service Discovery (Eureka or Consul).
- Inter-service communication (REST calls using `RestTemplate` / `WebClient`).

Database

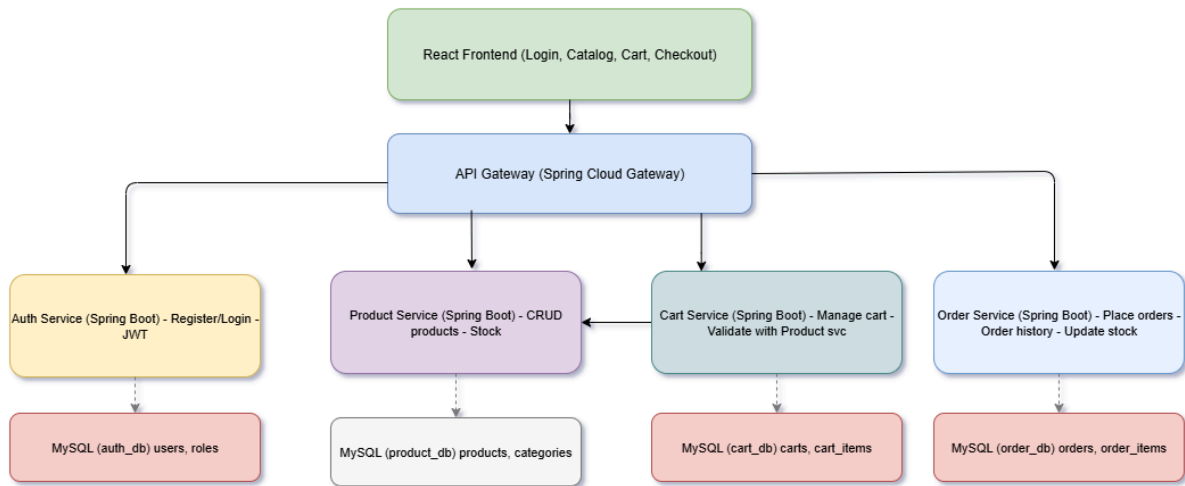
- MySQL basics (tables, relationships, queries).
- Designing schemas per service (auth_db, product_db, etc.).

Frontend (React)

- React basics (components, state, props).
- React Router (for navigation).
- Fetch API / Axios (for calling backend).
- JWT handling in frontend (store token, send in headers).

Out of Scope (for future releases)

- Docker (for containerization).
- Kafka/RabbitMQ (for async events).
- CI/CD basics (Jenkins/GitHub Actions).



Notes: - All services expose REST APIs - JWT passed via Authorization header - Each service has its own MySQL schema - Optional: use Kafka for async events

1. Root Folder

ecommerce-microservices/

- | — api-gateway/ # Spring Cloud Gateway project
- | — service-discovery/ # Eureka Server (or Consul)
- | — auth-service/ # Authentication + JWT
- | — product-service/ # Product catalog & stock
- | — cart-service/ # User cart
- | — order-service/ # Orders & checkout
- | — common-library/ # (Optional) shared DTOs, utils
- | — frontend/ # React app
- | — README.md

2. Inside each microservice (e.g., `auth-service/`)

auth-service/

- | — src/main/java/com/ecommerce/auth/
 - | — controller/ # REST Controllers
 - | — service/ # Business logic
 - | — repository/ # JPA repositories
 - | — model/ # Entities (User, Role, etc.)
 - | — config/ # Security, JWT filter
 - | — dto/ # Request/Response DTOs
 - | — AuthServiceApp.java
- | — src/main/resources/
 - | — application.yml # Service-specific config
- | — pom.xml

3. API Gateway

api-gateway/

- | — src/main/java/com/ecommerce/gateway/
 - | — GatewayApplication.java
- | — src/main/resources/
 - | — application.yml # Routing rules for services
- | — pom.xml

4. Service Discovery (Eureka)

```
service-discovery/
├── src/main/java/com/ecommerce/discovery/
│   └── DiscoveryServerApp.java
├── src/main/resources/
│   └── application.yml
└── pom.xml
```

5. Frontend (React)

```
frontend/
├── src/
│   ├── components/    # UI components
│   ├── pages/         # Login, ProductList, Cart, Order
│   ├── services/      # API calls (axios)
│   ├── App.js
│   └── index.js
├── public/
└── package.json
```

6. Common Library (Optional)

```
common-library/
├── src/main/java/com/ecommerce/common/
│   ├── dto/           # Shared DTOs
│   └── utils/         # Common utilities
└── pom.xml
```

Domain-Level Database Design with Relations

1. Auth Service → **auth_db**

Tables & Columns

- **users** (id, username, email, password, created_at)
- **roles** (id, name)
- **user_roles** (user_id, role_id)

Relationships

- **users** ↔ **user_roles** → **1 : M** (one user can have many role mappings)
 - **roles** ↔ **user_roles** → **1 : M** (one role can be assigned to many users)
 - **users** ↔ **roles** → **M : M** (via user_roles)
-

2. Product Service → **product_db**

Tables & Columns

- **products** (id, name, description, price, stock, created_at, updated_at)

Relationships

- Standalone (no internal joins, but will be referenced by cart & order via API).
-

3. Cart Service → **cart_db**

Tables & Columns

- **carts** (id, user_id, created_at)
- **cart_items** (id, cart_id, product_id, quantity)

Relationships

- **carts** ↔ **cart_items** → **1 : M** (one cart can have many items)
 - **users** ↔ **carts** → **1 : 1** (one user has exactly one cart, linked via user_id)
 - **products** ↔ **cart_items** → **1 : M** (one product can appear in many carts)
(logical relation only, resolved via API, not FK in DB)
-

4. Order Service → **order_db**

Tables & Columns

- **orders** (id, user_id, total_amount, status, created_at)
- **order_items** (id, order_id, product_id, quantity, price)

Relationships

- **orders** ↔ **order_items** → **1 : M** (one order has many items)
 - **users** ↔ **orders** → **1 : M** (one user can place many orders, linked via user_id)
 - **products** ↔ **order_items** → **1 : M** (one product can be in many orders)
(logical relation only, resolved via API, not FK in DB)
-

Cross-Service Logical Relations

- **Auth Service → Cart Service:** `users.id` ↔ `cards.user_id` (1 : 1)
- **Auth Service → Order Service:** `users.id` ↔ `orders.user_id` (1 : M)
- **Product Service → Cart Service:** `products.id` ↔ `cart_items.product_id` (1 : M)
- **Product Service → Order Service:** `products.id` ↔ `order_items.product_id` (1 : M)

-- auth_db

-- Tables :-

-- users : stores user details

-- roles : stores available roles (USER, ADMIN)

-- user_roles : mapping between users & roles (many-to-many)

CREATE DATABASE auth_db;

USE auth_db;

-- Users table

```
CREATE TABLE users (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Roles table

```
CREATE TABLE roles (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50) UNIQUE NOT NULL  
);
```

-- User ↔ Role mapping

```
CREATE TABLE user_roles (  
    user_id BIGINT NOT NULL,  
    role_id BIGINT NOT NULL,  
    PRIMARY KEY (user_id, role_id),  
    FOREIGN KEY (user_id) REFERENCES users(id),  
    FOREIGN KEY (role_id) REFERENCES roles(id)  
);
```

-- Insert default roles

```
INSERT INTO roles (name) VALUES ('ROLE_USER'), ('ROLE_ADMIN'), ('SERVICE');
```

-- product_db

-- Tables :-

-- products – list of available products

CREATE DATABASE product_db;

USE product_db;

CREATE TABLE products (

id BIGINT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

description TEXT,

price DECIMAL(10,2) NOT NULL,

stock INT NOT NULL,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE

CURRENT_TIMESTAMP

);

-- Example product

INSERT INTO products (name, description, price, stock)

VALUES ('iPhone 15', 'Latest Apple iPhone', 79999.99, 50);

-- cart_db

-- Tables

-- carts – one cart per user

-- cart_items – items inside the cart (linked to product)

```
CREATE DATABASE cart_db;
```

```
USE cart_db;
```

-- Cart linked to a user (from auth-service via user_id)

```
CREATE TABLE carts (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Items inside the cart

```
CREATE TABLE cart_items (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  cart_id BIGINT NOT NULL,  
  product_id BIGINT NOT NULL,  
  quantity INT NOT NULL,  
  FOREIGN KEY (cart_id) REFERENCES carts(id)  
  -- product_id is from product-service (validated via API, not FK here)  
);
```

-- Example cart

```
INSERT INTO carts (user_id) VALUES (1);
```

-- Example cart item

```
INSERT INTO cart_items (cart_id, product_id, quantity) VALUES (1, 1, 2);
```

-- order_db

-- Tables

-- orders – order info

-- order_items – items in an order

```
CREATE DATABASE order_db;
```

```
USE order_db;
```

-- Orders table

```
CREATE TABLE orders (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    status VARCHAR(20) DEFAULT 'PENDING',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Order items

```
CREATE TABLE order_items (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    order_id BIGINT NOT NULL,  
    product_id BIGINT NOT NULL,  
    quantity INT NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES orders(id)  
);
```

-- Example order

```
INSERT INTO orders (user_id, total_amount, status)  
VALUES (1, 159999.98, 'PENDING');
```

```
INSERT INTO order_items (order_id, product_id, quantity, price)  
VALUES (1, 1, 2, 79999.99);
```



```
-- Make sure user exists
CREATE USER IF NOT EXISTS 'ecom_user'@'localhost' IDENTIFIED BY 'ecom_pass';
-- Grant access only to all db
GRANT ALL PRIVILEGES ON auth_db.* TO 'ecom_user'@'localhost';
GRANT ALL PRIVILEGES ON product_db.* TO 'ecom_user'@'localhost';
GRANT ALL PRIVILEGES ON cart_db.* TO 'ecom_user'@'localhost';
GRANT ALL PRIVILEGES ON order_db.* TO 'ecom_user'@'localhost';
```

-- payment db

-- Create database

```
CREATE DATABASE IF NOT EXISTS payment_db;
```

```
USE payment_db;
```

-- Make sure user exists

```
CREATE USER IF NOT EXISTS 'ecom_user'@'localhost' IDENTIFIED BY 'ecom_pass';
```

-- Grant access only to all db

```
GRANT ALL PRIVILEGES ON payment_db.* TO 'ecom_user'@'localhost';
```

-- Payments table

```
CREATE TABLE payments (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  order_id BIGINT NOT NULL,  
  user_id BIGINT NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  currency VARCHAR(10) DEFAULT 'usd',  
  status VARCHAR(50) NOT NULL, -- e.g., PENDING, SUCCESS, FAILED  
  stripe_payment_intent_id VARCHAR(100), -- store Stripe's PaymentIntent ID  
  stripe_session_id VARCHAR(100), -- store Checkout Session ID  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
);
```

-- Indexes for faster lookups

```
CREATE INDEX idx_order_id ON payments(order_id);
```

```
CREATE INDEX idx_user_id ON payments(user_id);
```

✓ Auth-Service Completion Checklist

1. Core APIs

- `/register` → create user with encoded password, assign roles ✓
- `/login` → authenticate user, generate JWT ✓

2. Security

- JWT generation (`JwtUtils`) ✓
- JWT filter (Spring Security `OncePerRequestFilter`) → validates tokens ?
(*you should confirm this is present*)
- Password encoding (`BCryptPasswordEncoder`) ✓

3. Database

- `users`, `roles`, `user_roles` ✓
- Default roles inserted: `ROLE_USER`, `ROLE_ADMIN` ✓ (after our earlier fix)

4. DTOs

- `RegisterRequest`, `LoginRequest`, `JwtResponse` ✓

5. Error Handling

- Duplicate username/email handled ✓
Invalid login → Spring Security exception (already covered by `AuthenticationManager`) ✓

6. Return Data

- On login: return JWT + user info + roles ✓
- On register: success/failure message ✓

Future Scope

- **Refresh token API** (so JWT expiry can be renewed).
- **Logout API** (if you want token invalidation mechanism).
- **Email verification** (for production-grade apps).
- **Password reset** flow.

REST API Endpoints

- `/api/auth/register` -> registers a user

Request body

```
{
  "username": "testuser",
  "email": "testuser@example.com",
  "password": "password123",
  "role": ["user"]
}
```

Response Body

User registered successfully

- `/api/auth/login`

Request body

```
{
  "username": "testuser",
  "password": "password123"
}
```

Response Body

```
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ0ZXN0dXNlciIsImIhdCI6MTc1ODU1MDQ1MCwiZXhwIjoxNzU4NjM2ODUwfQ.gVE9qxWuksCj_sQFZA-j-v6jwDV5Ach9swqHW0Fb6p8",
  "type": "Bearer",
  "id": 1,
  "username": "testuser",
  "email": "testuser@example.com",
  "roles": [
    "ROLE_USER"
  ]
}
```

Flow:

1. Payment-Service produces event to Kafka topic `payment-topic` after payment success:

```
{
  "orderId": "1",
  "status": "PAID",
  "amount": "5000"
}
```

2. Order-Service Kafka listener (`OrderEventListener`) consumes the message automatically.
3. `OrderService.updateOrderStatus()` updates the database.
4. `/update/{orderId}` endpoint is still available if you need manual updates.

Kafka Set-up:

1. Download binary https://archive.apache.org/dist/kafka/3.2.1/kafka_2.13-3.2.1.tgz and extract
2. Update path in `C:\Kafka\kafka_2.13-3.2.1\config\server.properties`
`log.dirs=C:/Kafka/kafka_2.13-3.2.1/kafka-logs` to updated kafka path
3. Update path in `C:\Kafka\kafka_2.13-3.2.1\config\zookeeper.properties`
`dataDir=C:/Kafka/kafka_2.13-3.2.1/zookeeper-data`
4. Start zookeeper server in cmd at kafka path
`.\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties`
5. In separate cmd window start kafka server
`.\bin\windows\kafka-server-start.bat .\config\server.properties`
6. In separate cmd window create a new topic
`kafka-topics.bat --create --topic payment-events --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1`
Check if created
`kafka-topics.bat --list --bootstrap-server localhost:9092`
7. Producer will produce data(payment-service) and consumer will consume data(consumer-service)
To verify if data is getting produced from payment service, check in consumer(or directly in order-service)
`kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic payment-events --from-beginning`